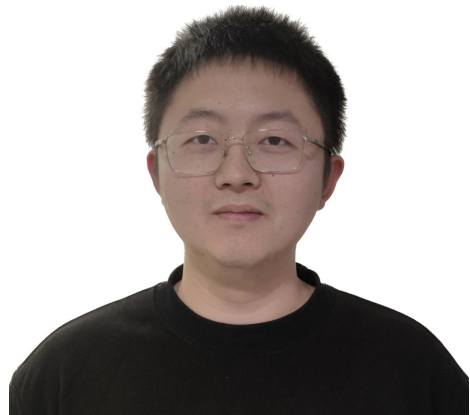




抖音内存优化实践

潘林林

自我介绍



潘林林

2016年本科毕业，在毕业之前和初期主导和参与多款 App 开发，14年便有独立制作的 App 上传至应用市场并取得累计超过10w次的下载量。在17年之后进入手机厂商工作，前后参与系统和相机子系统的稳定性和性能优化，21年初进入字节参与质量方向稳定性和内存的优化。

- 
- 01 背景介绍
 - 02 保障稳定性
 - 03 提升性能
 - 04 总结与展望

01

背景介绍

1. 内存优化必要性
2. 内存的定义和特点
3. 内存优化的挑战
4. 内存问题拆解

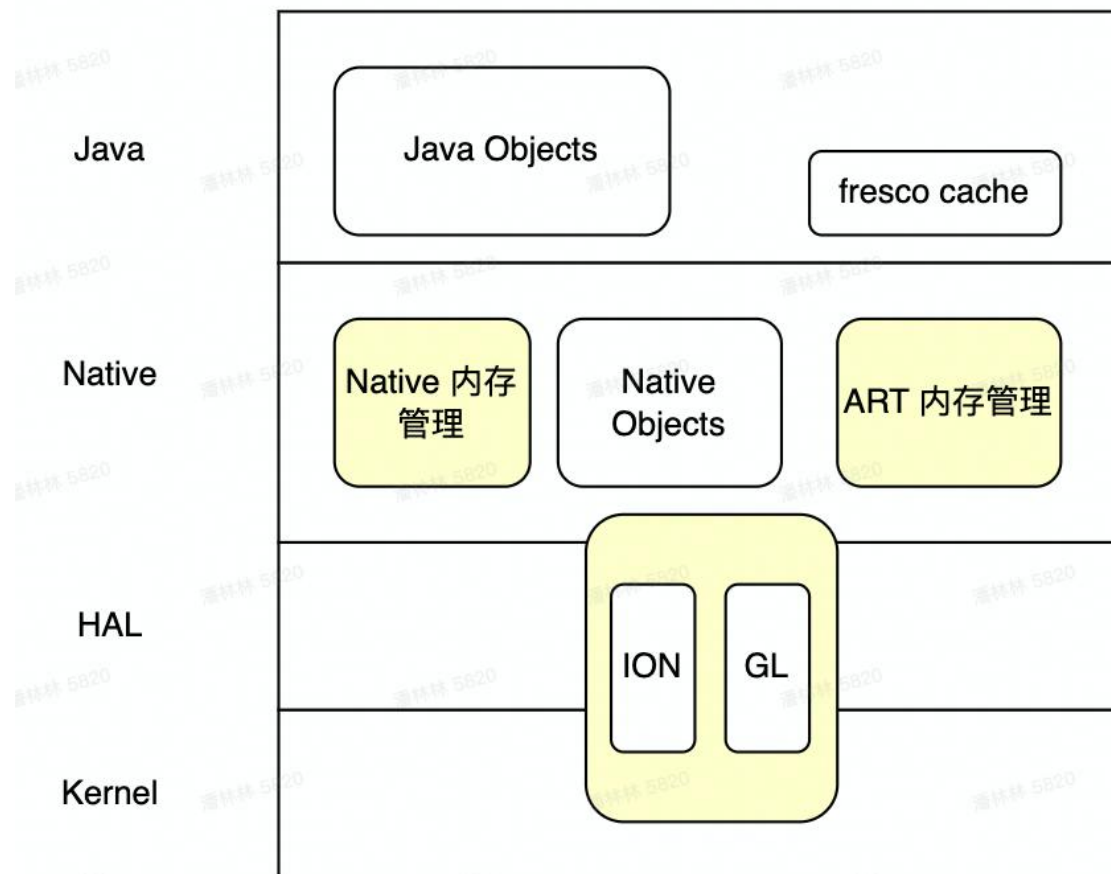
01_背景介绍：内存优化的必要性

1. 抖音新增业务需求，引入内存增长和泄漏，导致整体崩溃+20% ANR+5%
2. 抖音单个内存问题导致的崩溃，占比大盘总崩溃量的30%+
3. 优化不合理的内存使用，取得超出预期的投稿收益，相当于技术手段半年优化的总和
4. 优化高频对象的使用，带来了较大留存收益
- ...

内存问题能带来大幅的稳定性的劣化，优化也能带来超出预期的优化效果

01_背景介绍：内存的定义和特点

ByteDance



内存的定义：内存是内容的载体

内存的特点：

1. 内存和业务模块强相关
2. 内存的使用是全局的
3. 内存能在时间维度上积累
4. 内存类型多样，用户空间存在不同类型的内存管理器

挑战和机遇并存

01_背景介绍：内存优化的挑战和机遇

挑战

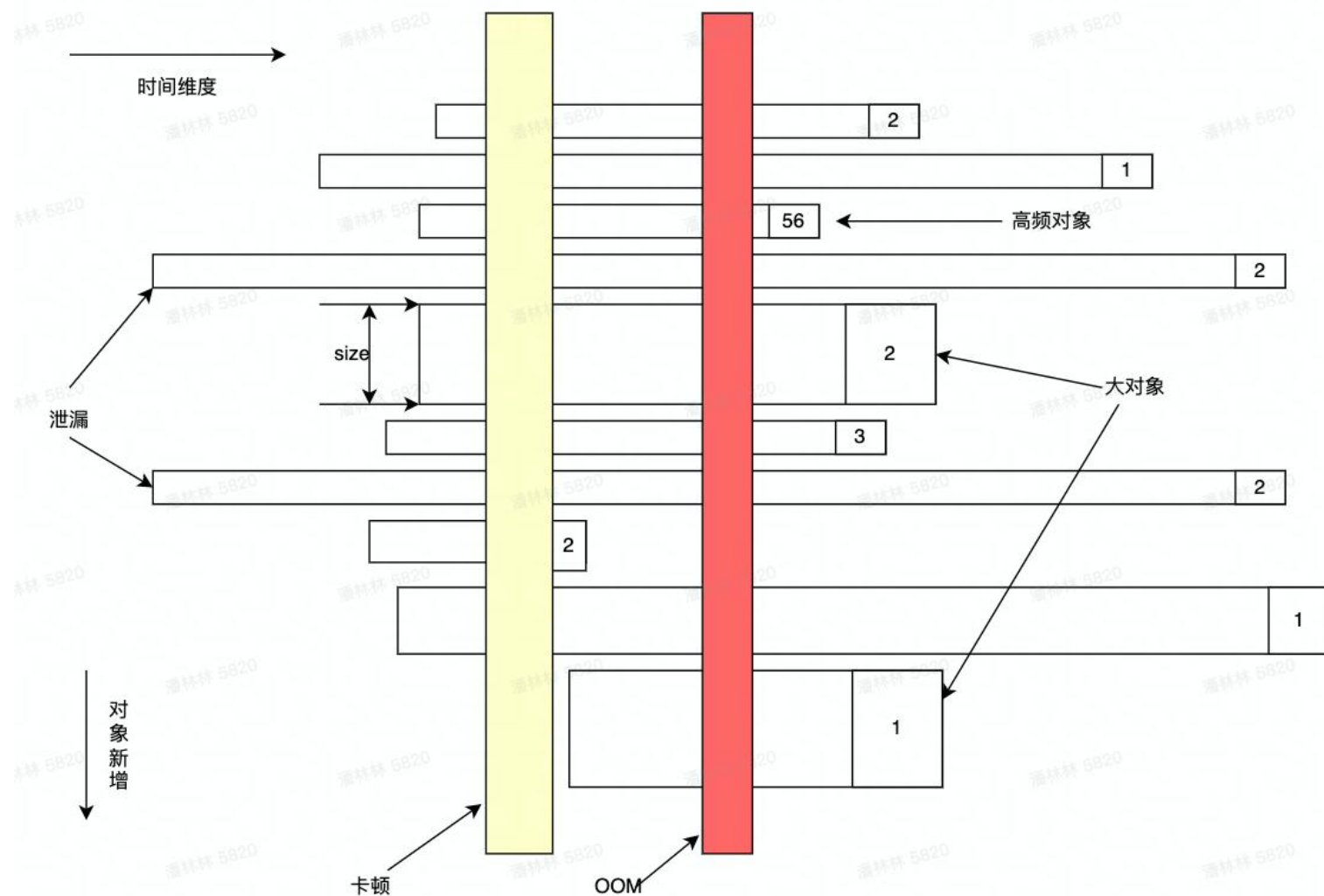
1. 内存优化需要深入了解业务模块，很多时候优化意味着解耦
 2. 内存使用全局且可累积，增加了问题回溯的复杂性
 3. 内存种类多，每种内存类型对应不同的资源管理方式，对工具和分析要求高
 4. 内存优化方式多样，不能一味的降低内存
- ...

机遇

1. 载体的特点有利于提升空间局部性
 2. 载体 + 时间维度可以均衡CPU使用
 3. 用户空间多种内存管理器给内存通用内存优化探索奠定基础
- ...

直面挑战，进一步对内存问题进行拆解

01_背景介绍：内存问题拆解



内存三要素：

1. size
2. count
3. 存活时间

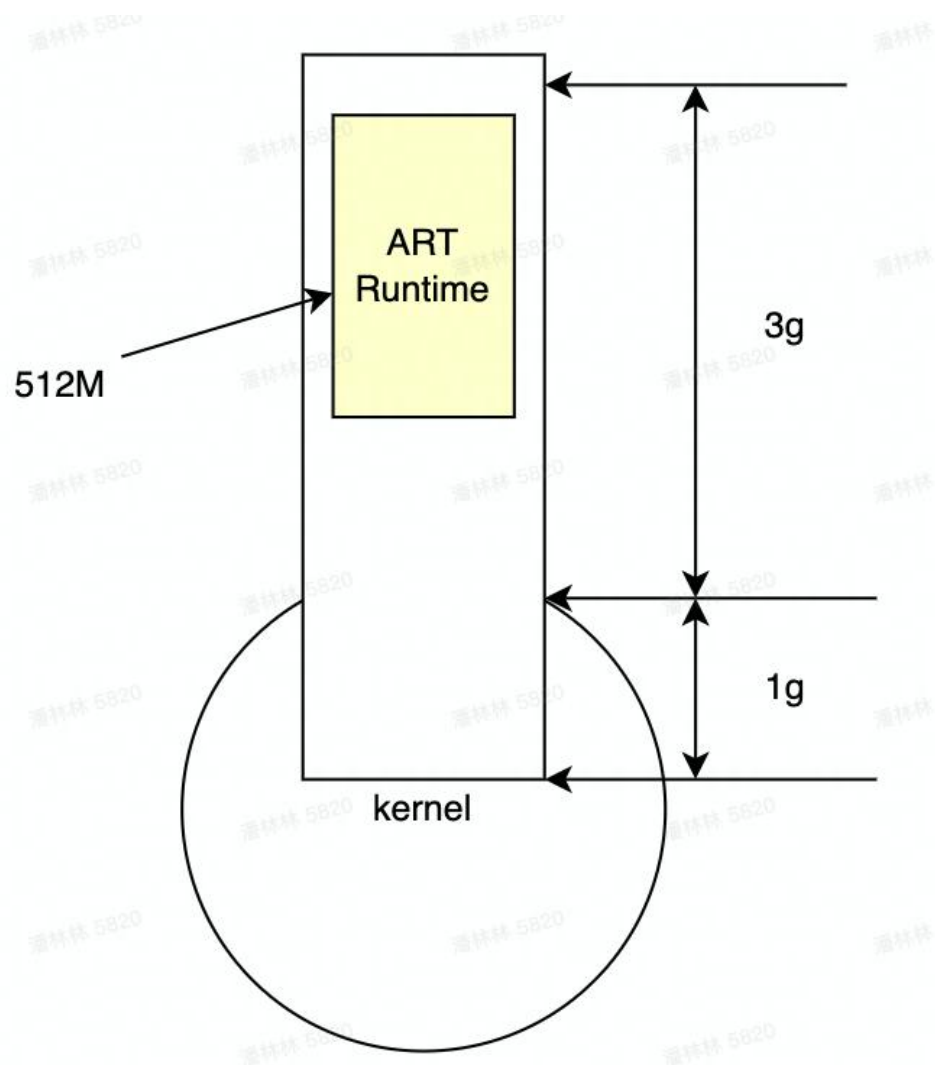
以上三要素有任一异常
均为出现了内存问题，
表现为稳定性和性能问题。

02

保障稳定性

1. OOM概述
2. 内存泄漏
3. 内存泄漏工具流程建设
4. 案例分享
5. 通用优化探索

02_ 保障稳定性：OOM概述



内存的限制被触发导致OOM

内存的限制来自：

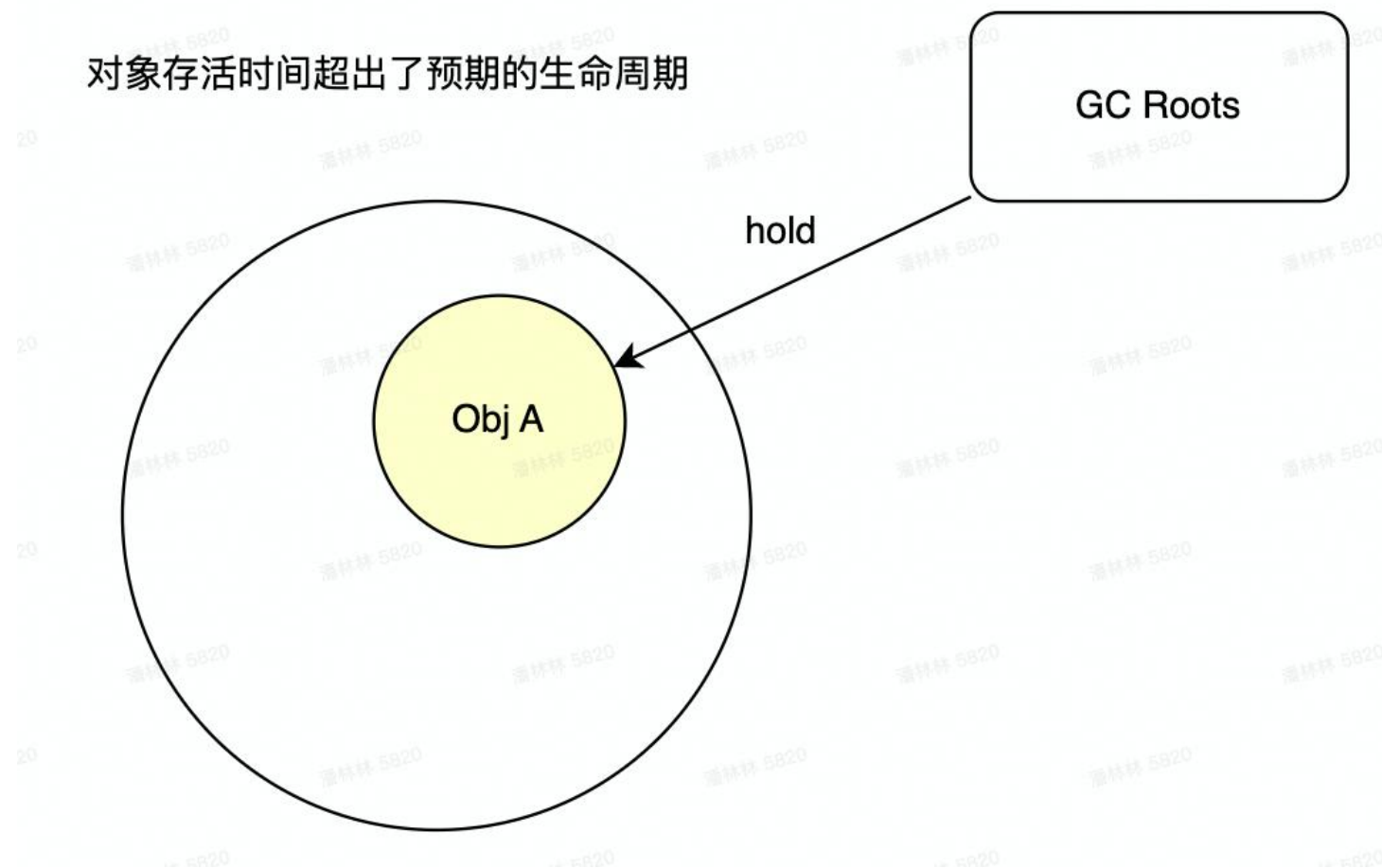
1. ART虚拟机的heap上限=> Java OOM
2. 进程的虚拟内存上限 => Native OOM
3. 驱动对内存的限制
4. 物理内存的上限

内存存活时间异常=> 内存泄漏 => OOM

内存size异常 => 内存峰值过高 => OOM

02_保障稳定性：内存泄漏

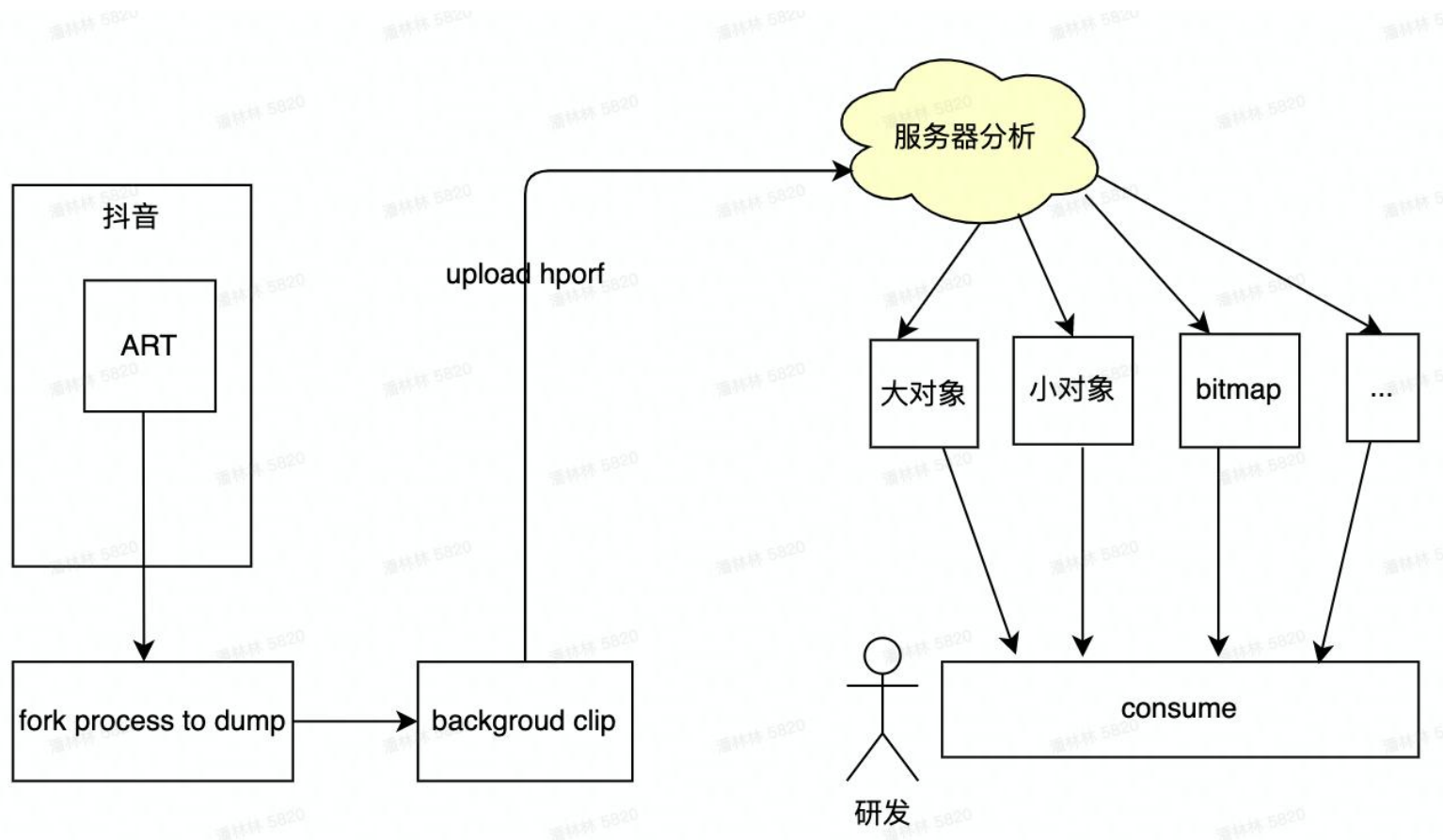
对象存活时间超出了预期的生命周期



常见GC ROOTs

1. Java/Native线程栈内引用的对象
2. 方法区中类静态属性、常量引用的对象
3. 所有被同步锁synchronized持有的对象

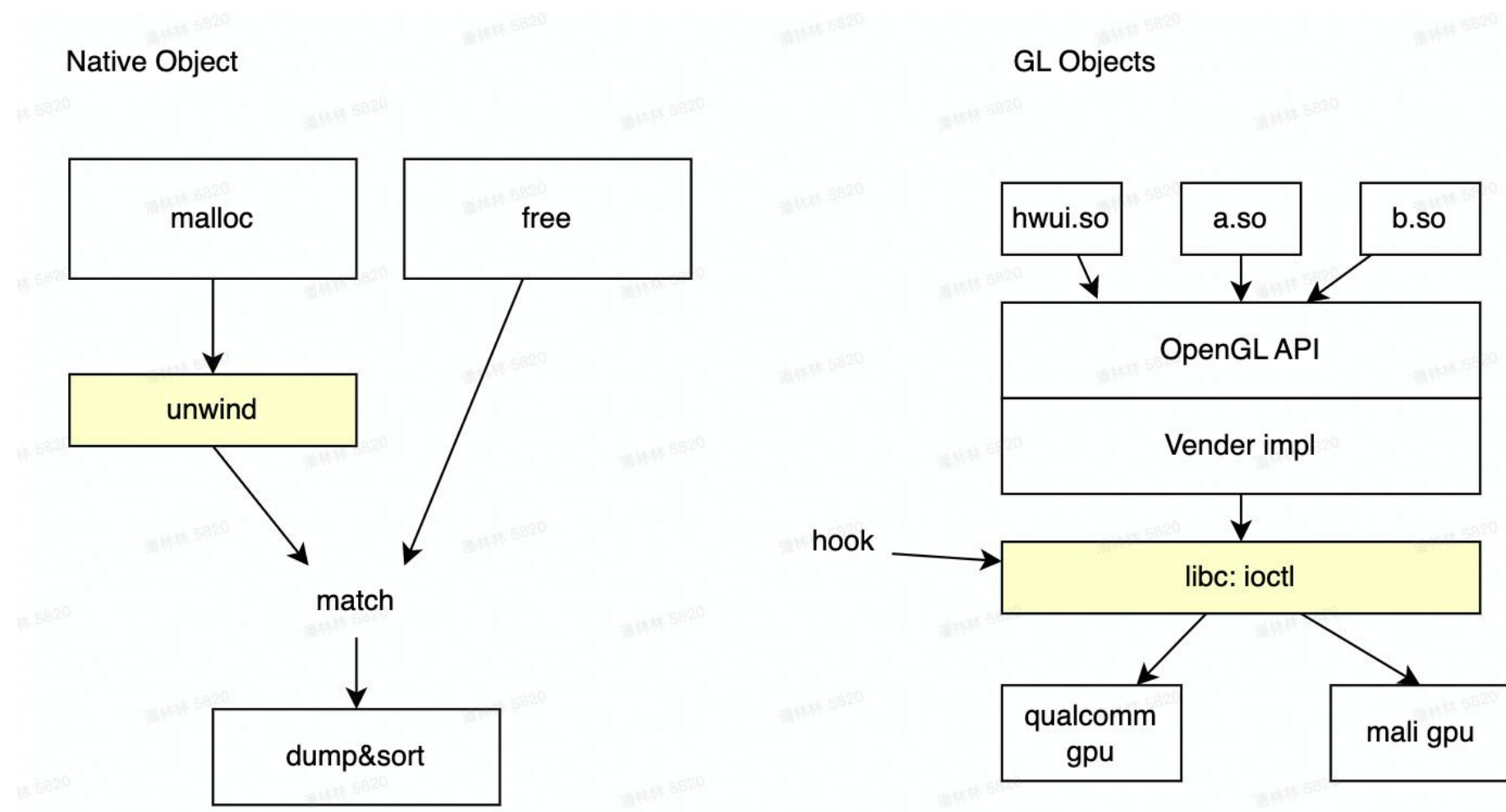
02_保障稳定性: Java内存泄漏检测



改进点:

1. 高性能dump
2. 监控对象拓展
3. 定制化自动分析
4. 消费闭环

02_保障稳定性: Native内存泄漏检测

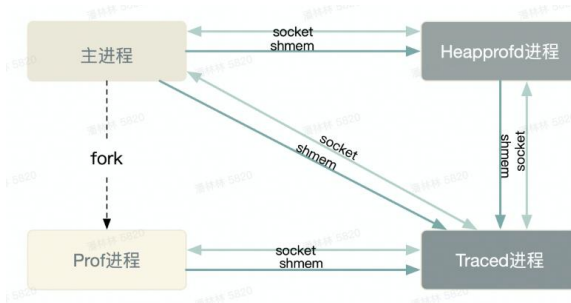


1. hook点确认
2. 高性能栈回溯
 1. 回溯动态库选取
 2. 限制回溯频次
 3. 减少无用栈回溯
 4. 重入问题解决
3. 数据匹配回捞消费

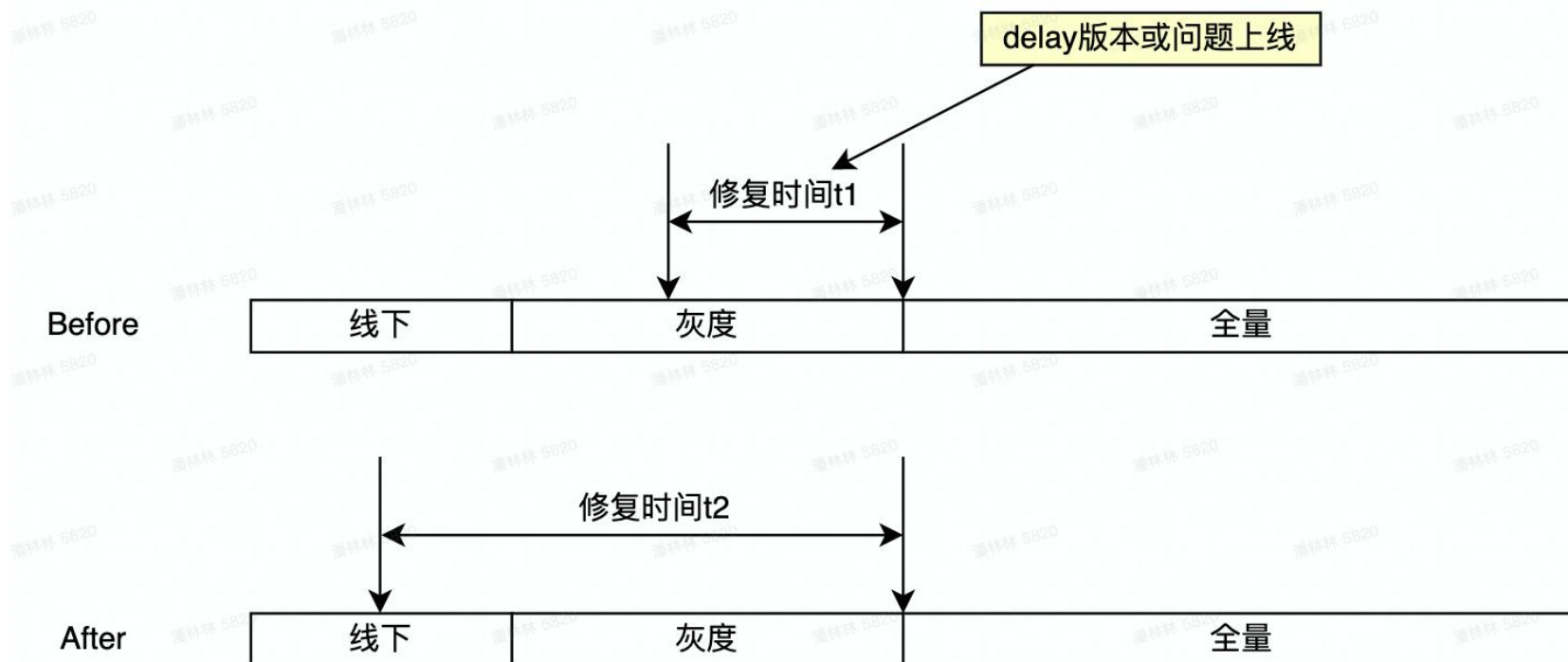
02_保障稳定性：线下内存防劣化

1. 内存长采集和分析能力建设

1. Prof进程使用多种接口进行profile
2. Heapprof接收数据进行聚合
3. Traced整体监控和数据输出



2. 问题左移效果：



02_保障稳定性：常规泄漏案例分享

```
public final ContextProviderFactory createDefaultContextProvider(IKitView iKitView, HybridBridge bridge) {
    IKitInitParam hybridParams;
    Intrinsic.checkNotNullParameter(iKitView, "iKitView");
    Intrinsic.checkNotNullParameter(bridge, "bridge");
    ContextProviderFactory contextProviderFactory = new ContextProviderFactory();
    View realView = iKitView.realView();
    View realView2 = iKitView.realView();
    HybridKitType type = null;
    Context context = realView2.getContext() != null ? realView2.getContext() : null;
    contextProviderFactory.setHybridContext(iKitView.getHybridContext());
    contextProviderFactory.registerWeakHolder(Context.class, context);
    contextProviderFactory.registerWeakHolder(View.class, realView);
    Context $this$apply = context != null ? m12237xae8b9cc6(context) : null;
    if ($this$apply instanceof Application) {
        contextProviderFactory.registerHolder(Application.class, $this$apply);
    }
    HybridContext hybridContext = contextProviderFactory.getHybridContext();
    if (!(hybridContext == null || (hybridParams = hybridContext.getHybridParams()) == null)) {
        type = hybridParams.getType();
    }
    contextProviderFactory.registerHolder(IKitInstanceApi.class, new LegacyKitInstance(new WeakReference(iKitView), iKitView,
        final /* synthetic */ HybridBridge $bridge$inlined;
        final /* synthetic */ Context $context$inlined;
        final /* synthetic */ IKitView $iKitView$inlined;
        final /* synthetic */ View $realView$inlined;

        /* JADX INFO: Access modifiers changed from: package-private */
        {
            this.$iKitView$inlined = r3;
            this.$context$inlined = r4;
            this.$realView$inlined = r5;
            this.$bridge$inlined = r6;
        }

        @Override // com.p152ss.android.ugc.aweme.hybridkit.bridge.LegacyKitInstance, com.bytedance.ies.bullet.core.kit.IKitIns
        public BulletKitType getKitType() {
            if (HybridKitType.this == HybridKitType.LYNX) {
                return BulletKitType.LYNX;
            }
            if (HybridKitType.this == HybridKitType.WEB) {
                return BulletKitType.WEB;
            }
        }
    }
```

常见泄漏情况：

1. 静态实例持有外部对象致使外部对象无法回收
2. 非静态内部类持有外部类对象，内部类对象生命周期超过外部类
3. 异步线程/callback持有外部对象
4. 集合类子项持有外部对象引用，未及时清除

02_保障稳定性：案例分享：size异常

Signal 6(SIGABRT), Code -6(SI_TKILL)

abort message: 'glTexImage2D error! GL_OUT_OF_MEMORY (0x505)'

#00 pc 0001a70e /system/lib/libc.so (abort+63)

#01 pc 000065a1 /system/lib/liblog.so (__android_log_assert+156)

#02 pc 0002fe31 /system/lib/libhwui.so (android::uirenderer::debug::GlesErrorCheckWrapper::assertNoErrors+192)

#03 pc 0002d7f7 /system/lib/libhwui.so (wrap_glTexImage2D+52)

#04 pc 000846b5 /system/lib/libhwui.so (android::uirenderer::uploadToTexture+104)

#05 pc 0008434f /system/lib/libhwui.so (android::uirenderer::Texture::upload+714)

#06 pc 00084bd3 /system/lib/libhwui.so (android::uirenderer::TextureCache::createTexture+114)

#07 pc 00084d0f /system/lib/libhwui.so (android::uirenderer::TextureCache::getCachedTexture+226)

#08 pc 0008518b /system/lib/libhwui.so (android::uirenderer::TextureCache::prefetchAndMarkInUse+6)

#09 pc 0004b869 /system/lib/libhwui.so (android::uirenderer::renderthread::OpenGLPipeline::pinImages+36)

#10 pc 0005ef87 /system/lib/libhwui.so (android::uirenderer::DisplayList::prepareListAndChildren+30)

#11 pc 00078f17 /system/lib/libhwui.so (android::uirenderer::RenderNode::prepareTreeImpl+182)

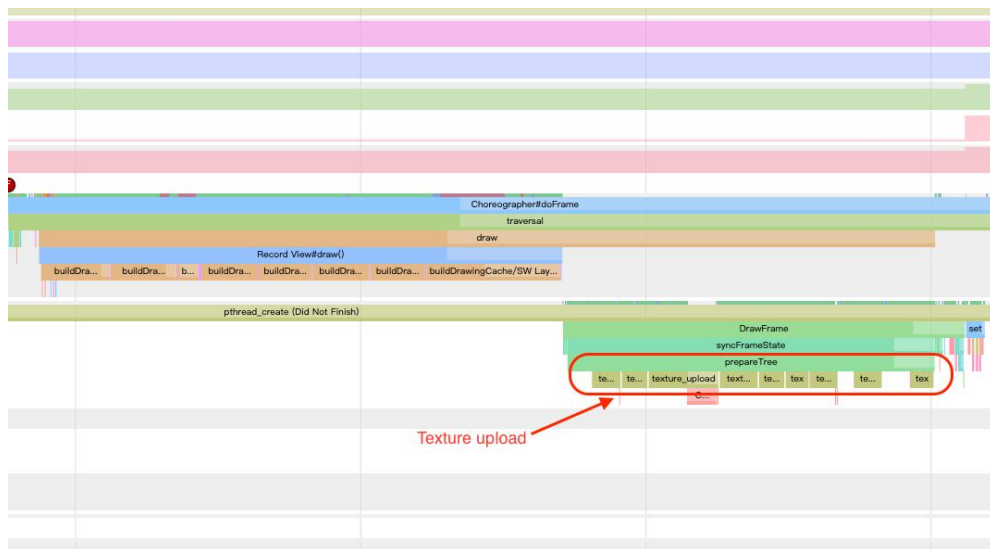
#12 pc 0003ebbl /system/lib/libhwui.so (std::__1::function::operator()+48)

#13 pc 0005efb1 /system/lib/libhwui.so (android::uirenderer::DisplayList::prepareListAndChildren+72)

#14 pc 00078f17 /system/lib/libhwui.so (android::uirenderer::RenderNode::prepareTreeImpl+182)

02_保障稳定性：案例分享：size异常

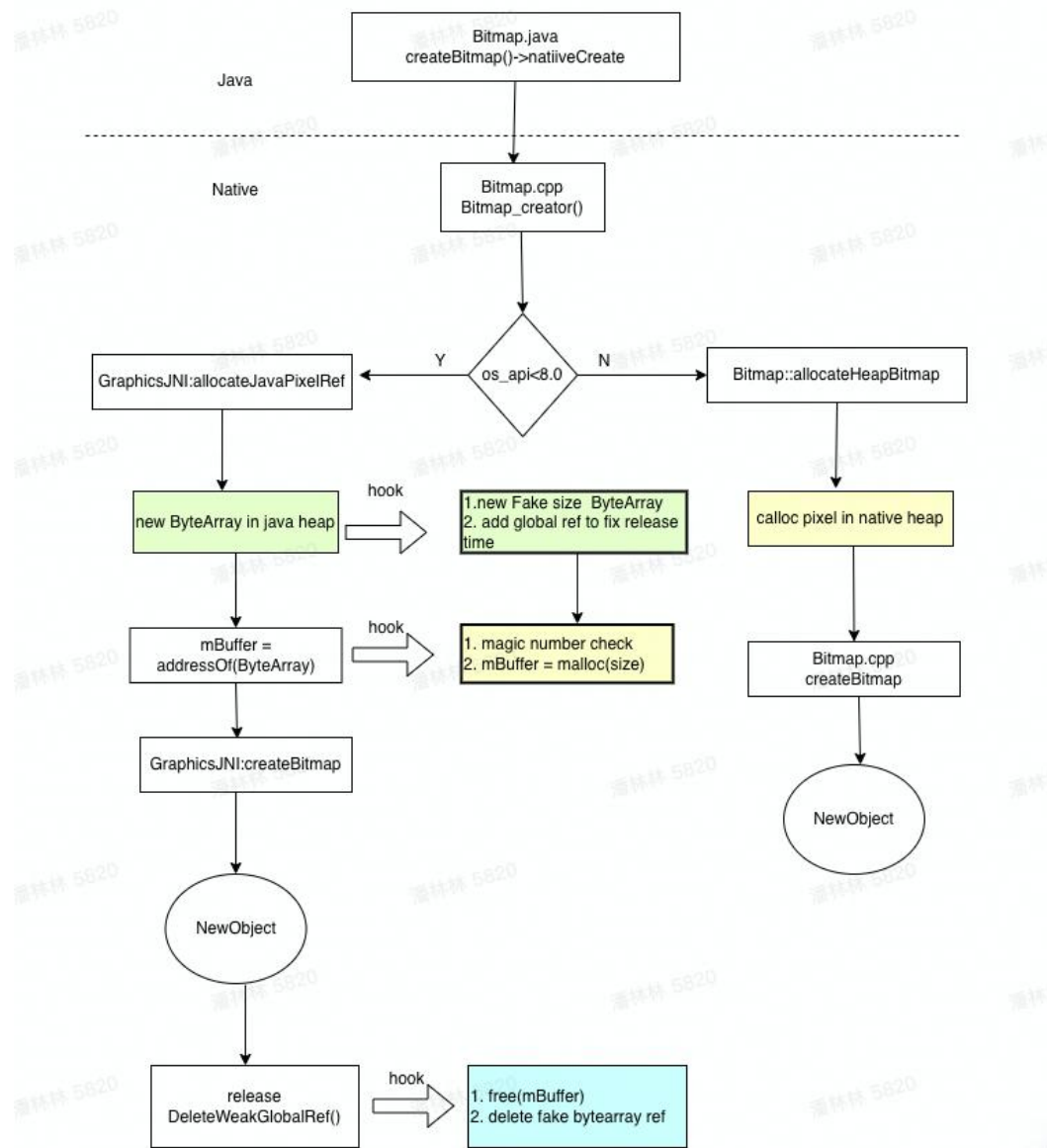
```
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9142272, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9142272, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9142272, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
Graphic_Mem: IOCTL mali ALLOC size:9342976, threadNameRenderThread
```



1. Hook gpu驱动的ioctl捕捉异常内存使用
2. 确定页面信息和问题发生时机
3. 源码层面补充埋入ATrace，定位具体的View
4. 最终定位相关view由于开启软件绘制生成bitmap缓存导致

02_保障稳定性：通用优化探索：Java OOM优化

NativeBitmap方案

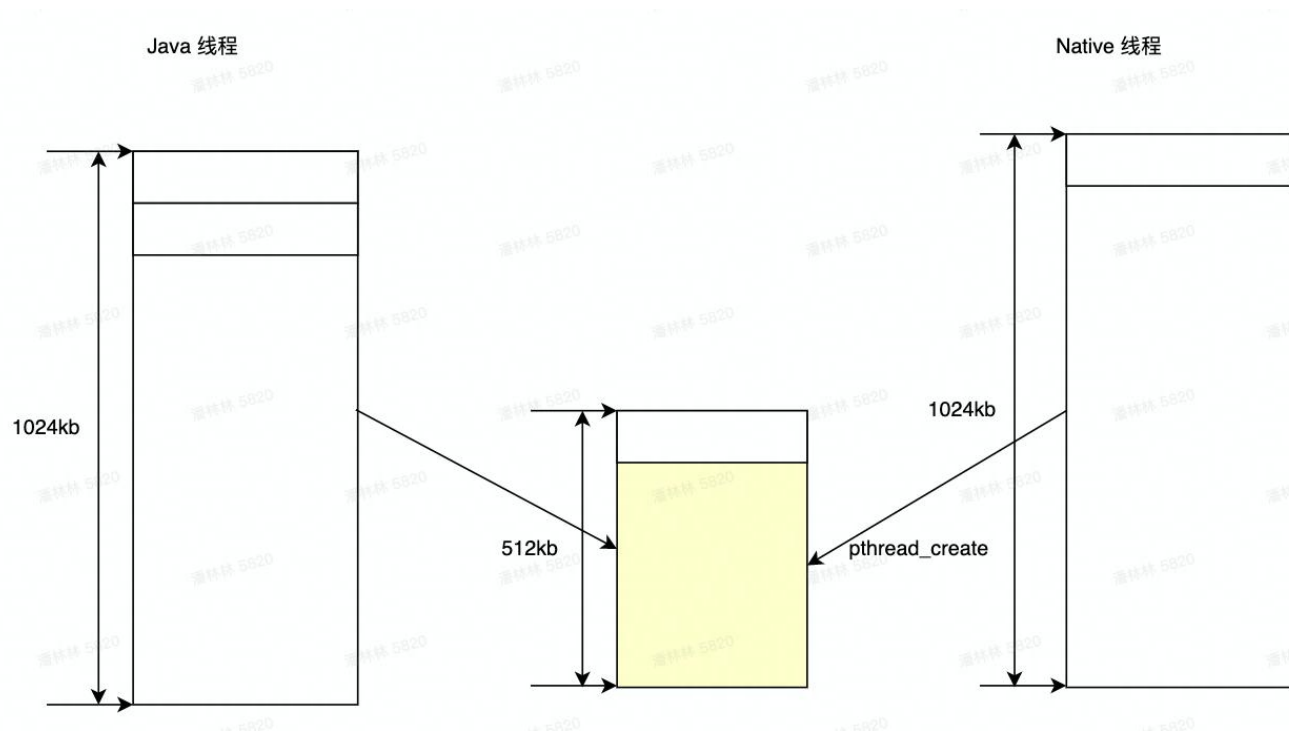


关键点:

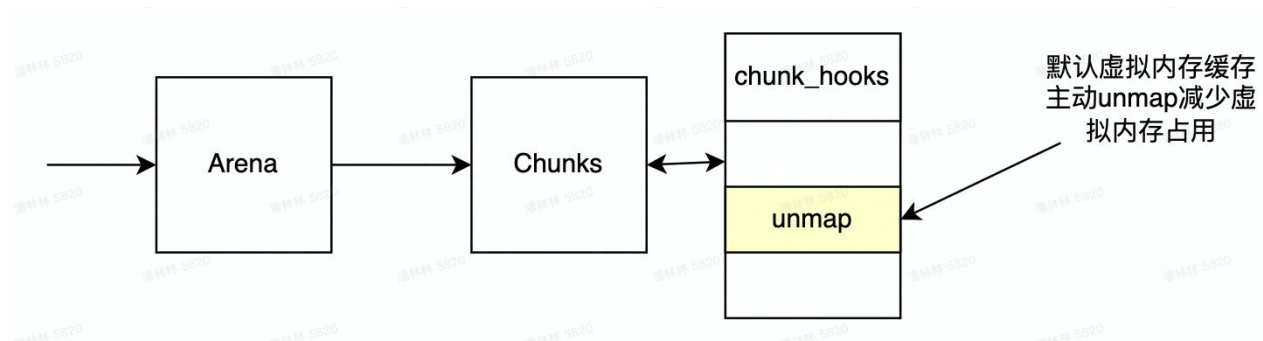
1. 原分配java内存的地方，分配空对象
2. Native层面完成实际的内存分配
3. 及时回收假对象和Native内存

02_ 保障稳定性：通用优化探索：Native OOM优化

线程栈裁剪



Chunk unmap



03

提升性能

1. 内存与性能的理论分析
2. 性能优化案例分享
3. 通用优化探索

03_提升性能：内存与性能的关联分析

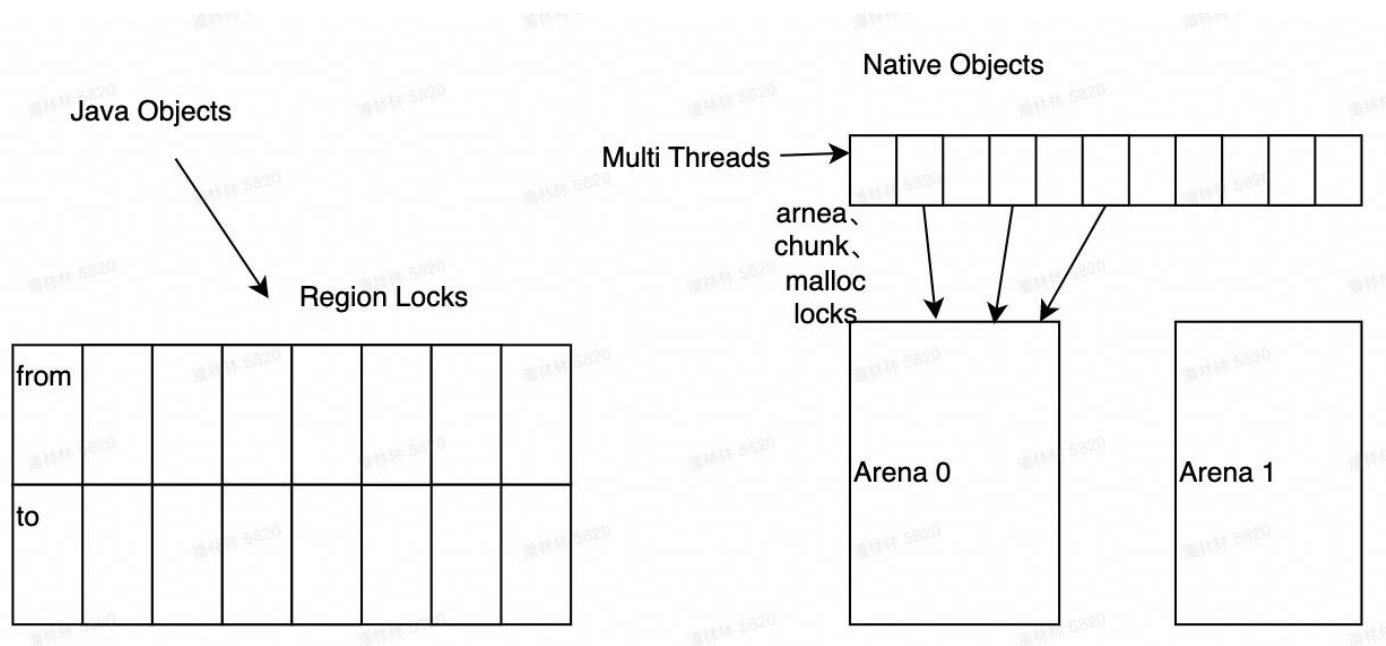
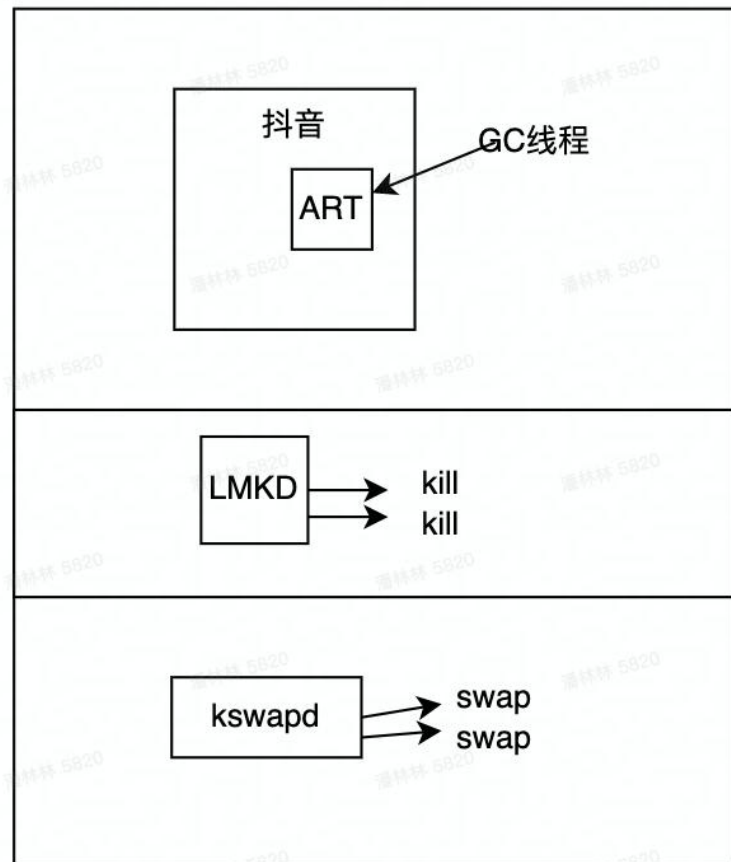
减：减少内存使用量，优化内存竞争和超限导致的性能和稳定性问题

1. 内存紧张导致内存申请持锁竞争
2. 内存使用过多导致内存回收线程抢占CPU
3. 高频重复的对象创建增加内存使用同时占用CPU时间

增：增加内存使用量，利用内存优化CPU使用率（平均值和峰值）

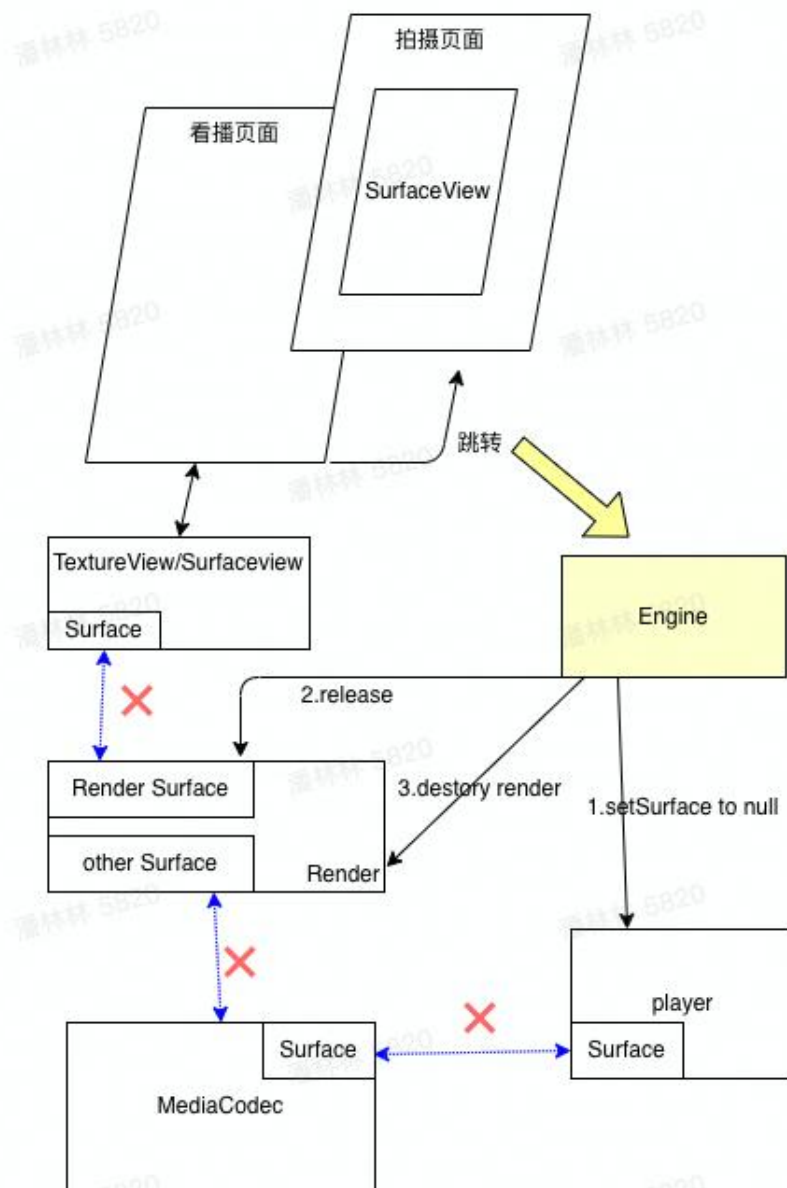
1. 局部性优化【缓存、复用、线程本地化存储】
2. 并发性优化【多线程减少内存锁竞争、内存size分类】
3. 负载均衡【预加载】

03_提升性能：内存与性能的关联



03_提升性能：案例分享：内存降级

播放器显存降级



关键点：

1. 切断所有surface引用
2. 清除核心render组件完成显存降级

03_提升性能：案例分享：高频对象

com.ss.android.ugc.aweme.compliance.business.recommend.RecommendSwitchManager.getPersonalRecommendStatusValue	150ms
com.ss.android.ugc.aweme.compliance.protection.serviceimpl.TeenModeServiceImpl.isTeenModeON	150ms
com.ss.android.ugc.aweme.compliance.protection.timelock.TimeLockRuler.isTeenModeON	150ms
com.ss.android.ugc.aweme.compliance.protection.teenmode.TeenageModeManager.getMTeenModeStatus	150ms
com.ss.android.ugc.aweme.compliance_protection_business.teen_mode.service.TeenModeBusinessServiceImpl.getTeenM	150ms
com.ss.android.ugc.aweme.compliance_protection_common_api.common.settings.ComplianceProtectionRefactorExpe	150ms

高频对象治理



通过慢函数发现热修插桩引入高频临时对象分配

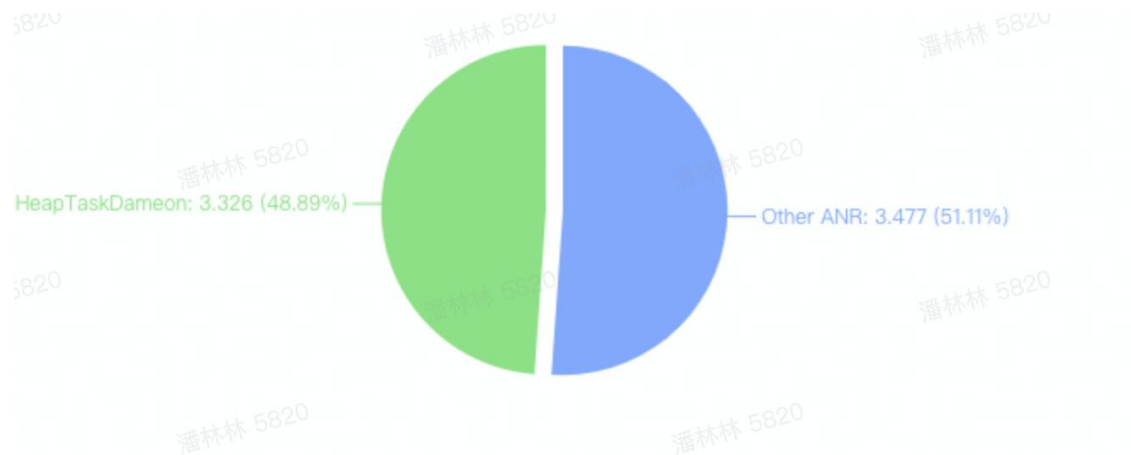


在高频调用中复用临时对象

```
public static PatchProxyResult proxy(Object[] objArr, Object obj, ChangeQuickRedirect ch.  
    PatchProxyResult patchProxyResult = new PatchProxyResult();  
    if (changeQuickRedirect == null) {  
        return patchProxyResult;  
    }
```

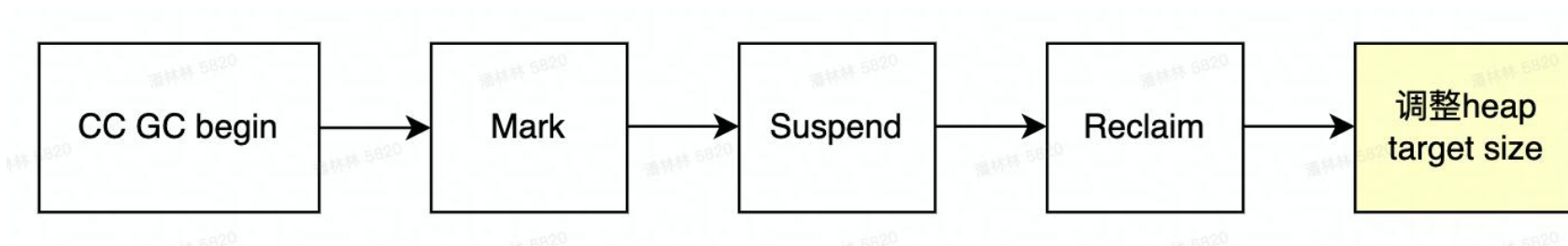
```
private static final PatchProxyResult EMPTY_RESULT = new PatchProxyResult();  
public static PatchProxyResult proxy(Object[] objArr, Object obj, ChangeQuickRedirect ch.  
    if(enable && changeQuickRedirect == null) {  
        return EMPTY_RESULT;  
    }
```

03_提升性能：通用优化1：GC降活



降低GC的次数 => 确定GC触发的条件 => 修改条件让GC相对不容易触发

03_提升性能：通用优化1：GC降活



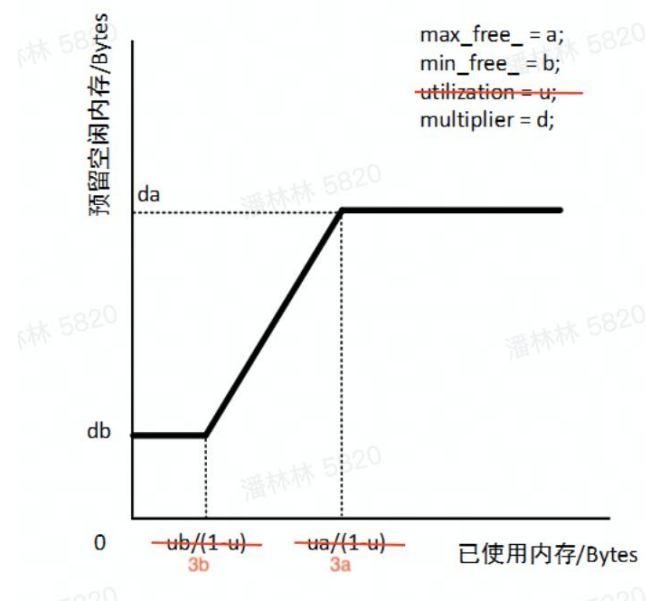
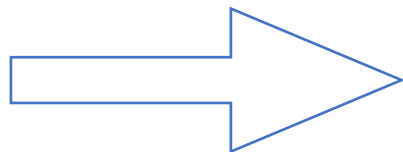
让GC不容易触发 = 增大 target size = 增大 (已经使用的内存 + 预留size*3)

=>

增大预留size

03_提升性能：通用优化1：GC降活

1. 已使用的内存 $< 3 * \text{min_free}$
预留size = $3 * \text{min_free}$
2. $3 * \text{min} < \text{已经使用的内存} < 3 * \text{max}$
预留size = 已经使用的内存
3. 已使用的内存 $> 3 * \text{max_free}$
预留size = $3 * \text{max_free}$



03_提升性能：通用优化1：GC降活

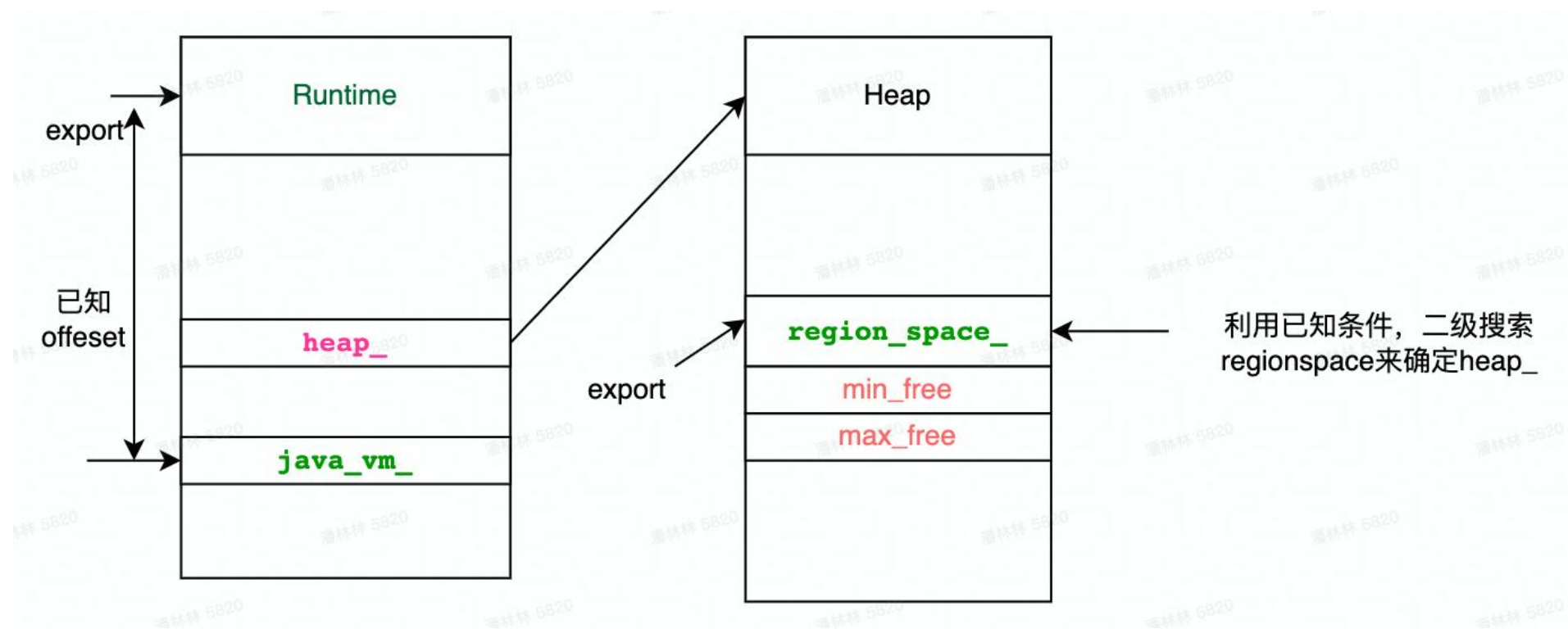
增大预留size = 增大max_free 或 min_free



增大max_free 或 min_free = 降低gc线程的活跃度

03_提升性能：通用优化1：GC降活

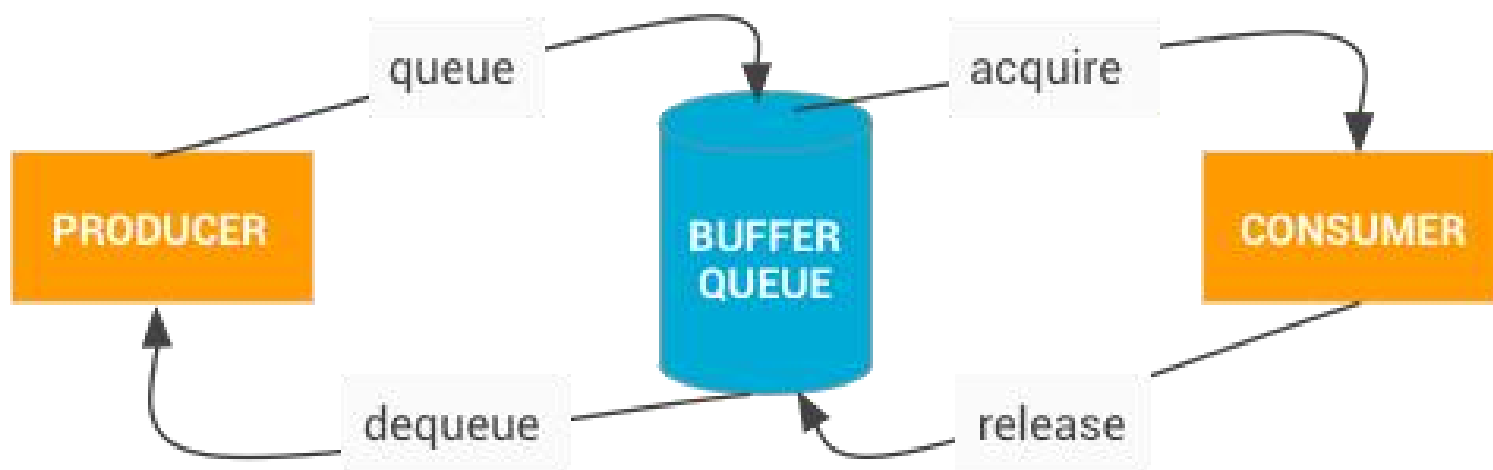
min_free max_free内存搜索流程



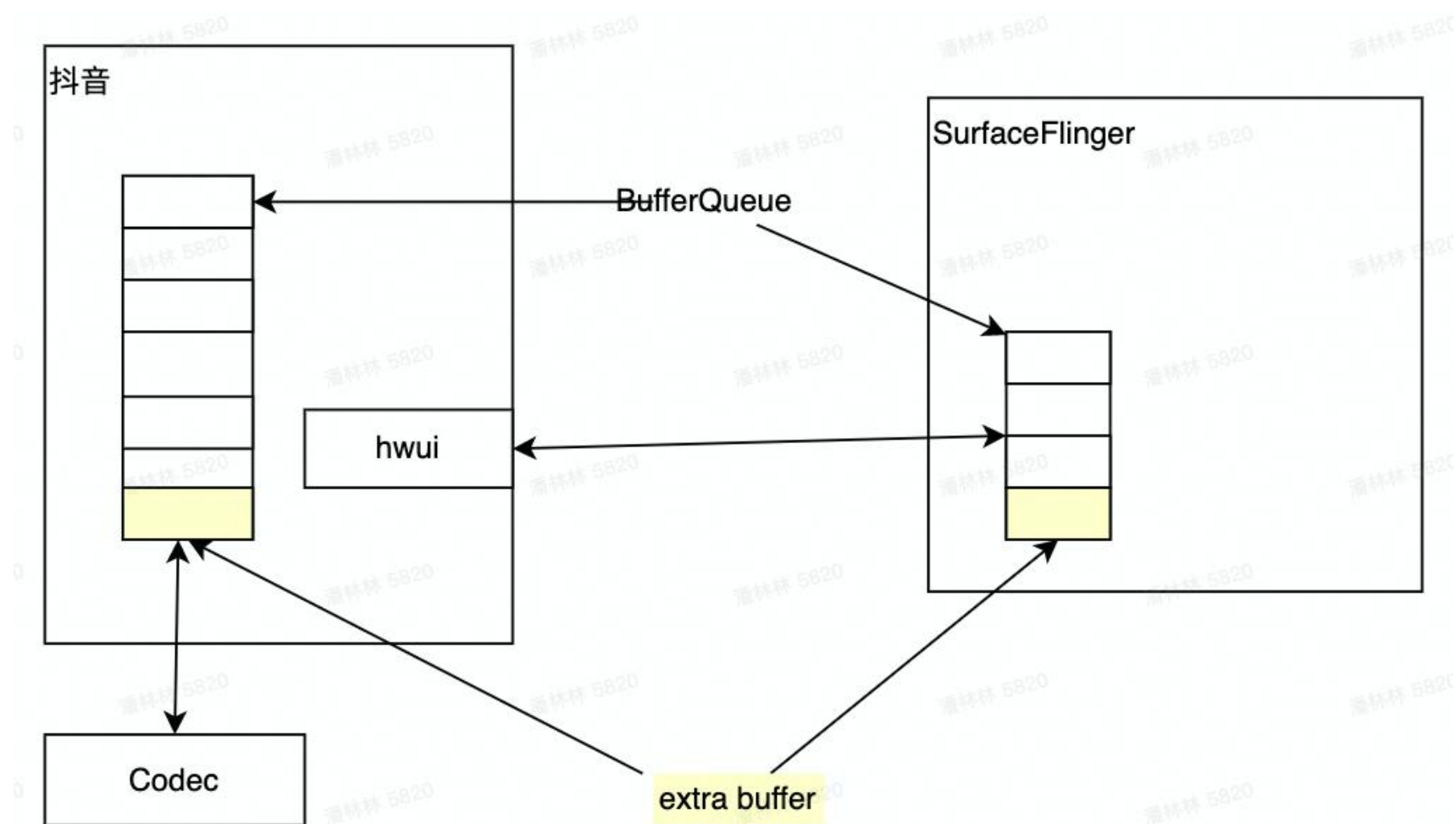
03_提升性能：通用优化1: GC降活效果

- 首刷时长p50 v2 -0.47% (-18.858ms)
- 大盘帧率 v2 +0.25% (+0.1294)
- 推荐页帧率 v2 +0.247%
- 推荐页丢帧次数 v2 -2.683%
- 首刷帧率 v2 +0.933%
- 前台电流均值 v2 -0.693%
- 前台CPU使用率均值 v2 -1.683%
- 前台电池温度平均值 v2 -0.0919%
- 人均活跃天数 v2 +0.0221%
- 按进组时间次留 v2 +0.0303%
- 人均app使用时长 v2 +0.0499%
- 人均播放时长 v2 +0.045%
- 人均vv数 v2 +0.0956%
- 人均session个数 v2 +0.134%
- 人均投稿数 v2 +0.214%
- 人均开拍次数 v2 +0.226%

03_提升性能：通用优化2：显存extra buffer



03_提升性能：通用优化2：显存extra buffer



BQ 总长度 =
maxDequeuedBufferCount
+
maxAcquiredBufferCount
+
? asyncMode 1 : 0

03_提升性能：通用优化2：显存extra buffer

1. max dequeued buffer count

```
status_t err = mGraphicBufferProducer->setMaxDequeuedBufferCount(  
    maxDequeuedBuffers);
```

2. max acquired buffer count

而max acquired buffer数量设定接口来自consumer，常规的GUI体系架构中，consumer在SurfaceFlinger进程，可控性较低且修改maxDequeuedBufferCount和maxAcquiredBufferCount的效果是一样的

3. Async mode

```
int maxBufferCount = mMaxAcquiredBufferCount + mMaxDequeuedBufferCount +  
    ((mAsyncMode || mDequeueBufferCannotBlock) ? 1 : 0);
```

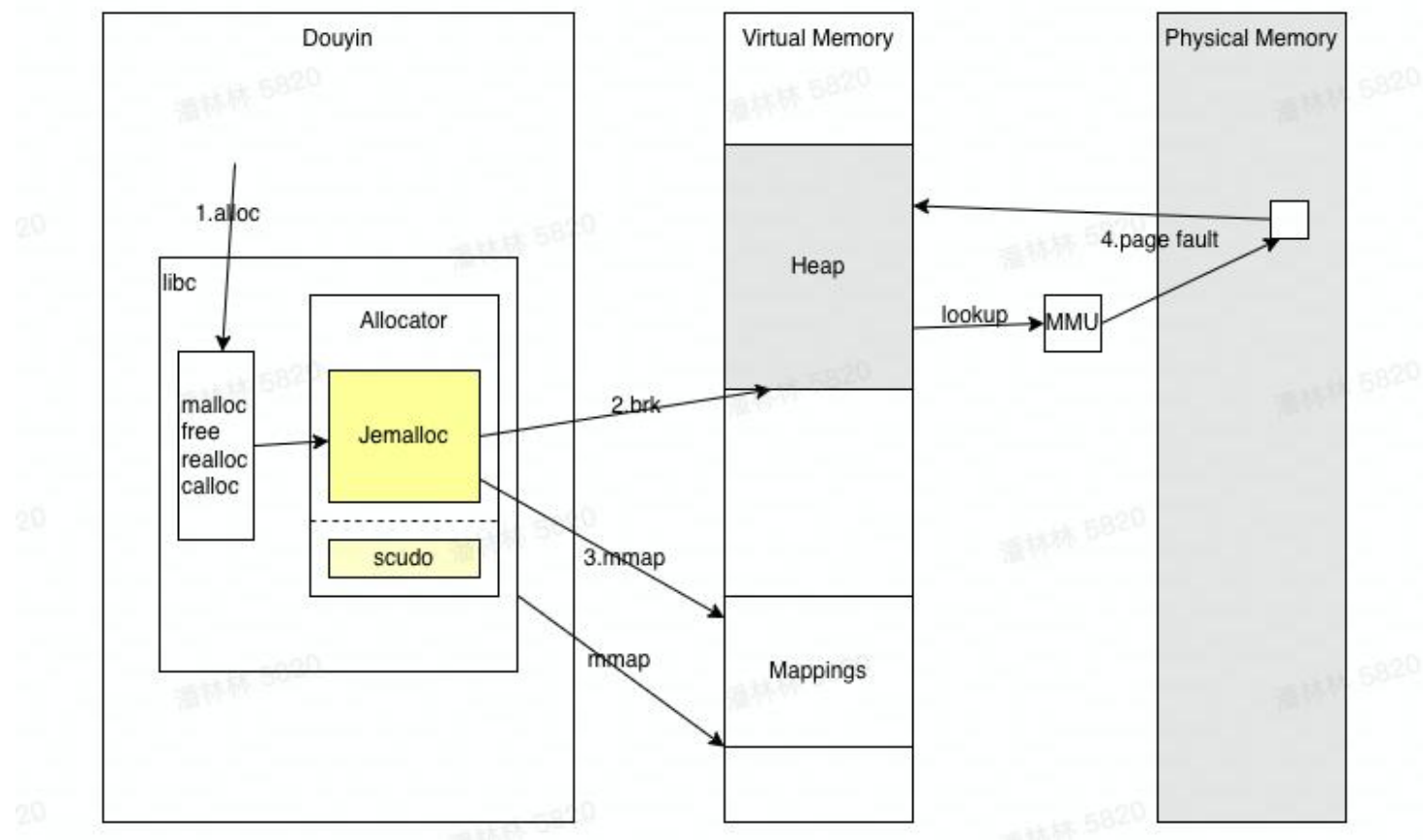
通过改变max dequeued buffer count 来改变BQ的长度，增加可轮转的buffer数量

03_提升性能：通用优化2：显存extra buffer效果

- 播放流畅度显著优化：
- 卡顿用户占比-0.6753%
- 单vv卡顿率-0.456%
- 单vv卡顿时长-0.458%
- 首刷启动：
- 冷启时长均值-0.846%
- 首刷时长均值-0.7%
- 留存指标波动，中端机LT30+0.053%
- 核心及基础指标波动
- 部分交互指标优化

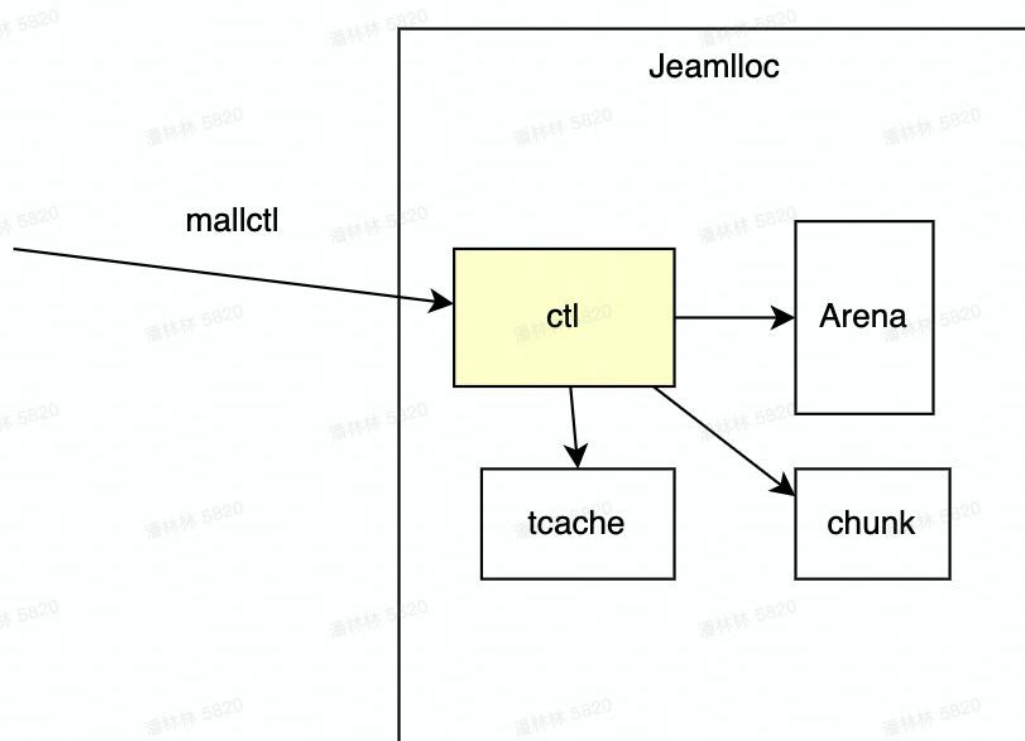
03_提升性能：通用优化3：Jemalloc动态调优

Native Allocator基本架构

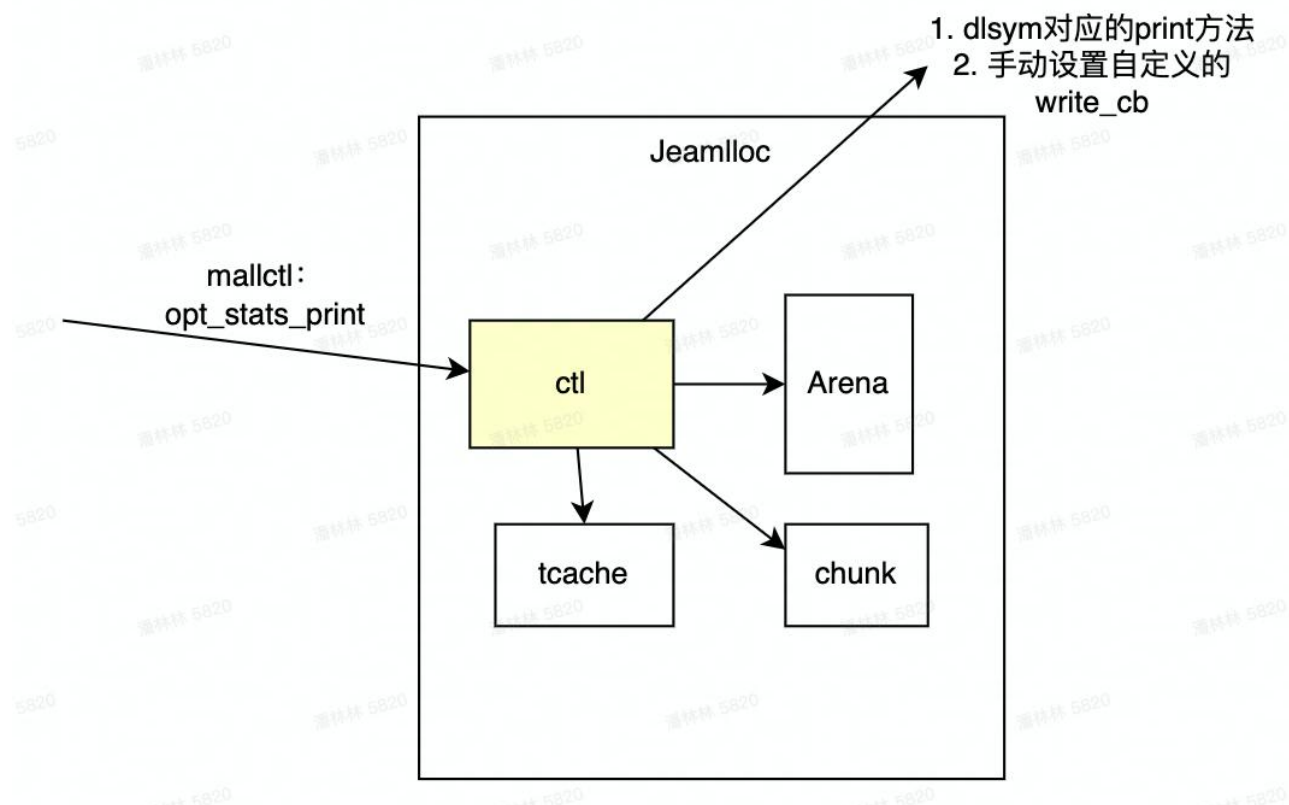


03_提升性能：通用优化3：Jemalloc动态调优

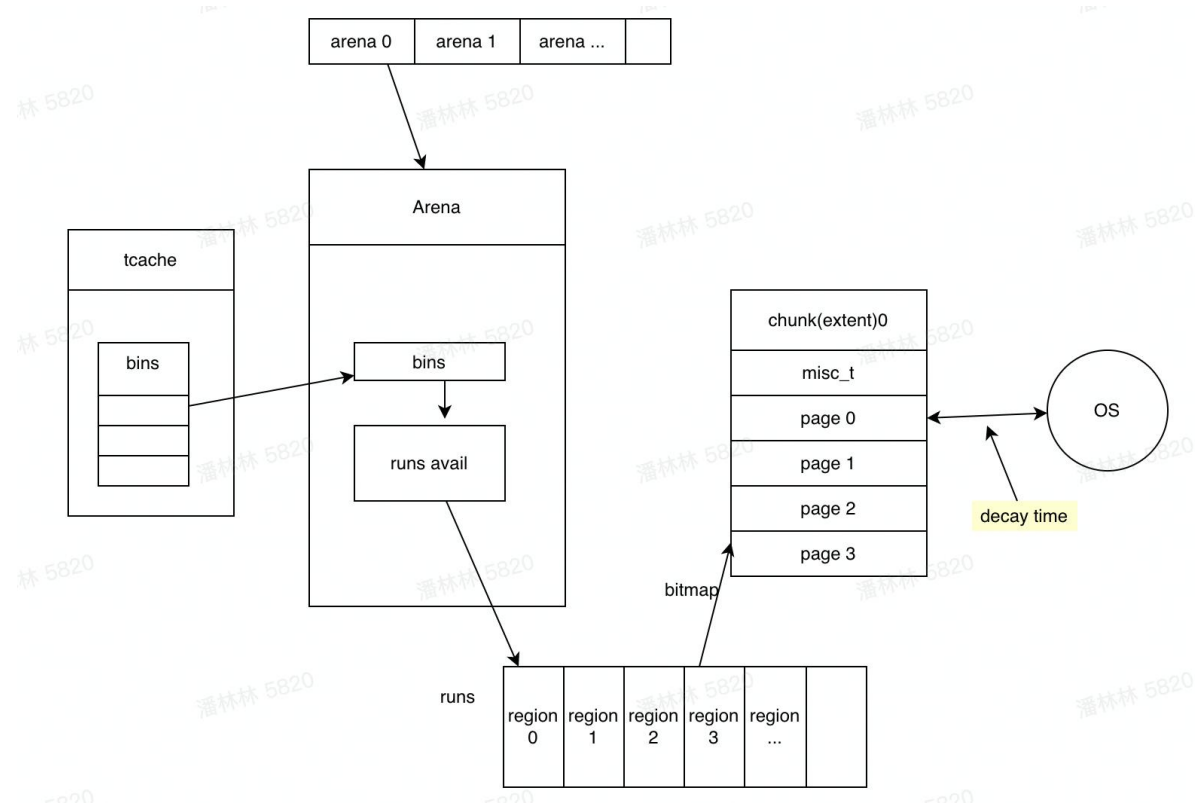
Jemalloc ctl 子系统



Jemalloc 状态输出



03_提升性能：通用优化3：Jemalloc动态调优



Decay time可以设定内存释放回系统的延迟时长，延长此时间，会在系统和**jemalloc**直接形成一定时间的内存缓存，有利于后续内存申请的性能提升。

03_提升性能：通用优化3: Jemalloc动态调优效果

- **FEED:**

- FEED滑动丢帧次数-3.069%,

- Feed-CPU使用率 0.585%,

- Feed-CPU使用速率 -0.353%

- **冷启首刷:**

- 启动时长均值-1.875% (-25.108ms),

- 首刷时长均值 -1.698%(-34.145ms)

- 留存指标 人均活跃天数 +0.0549%

- **核心及基础指标**

- 人均app使用时长+0.0846%,

- 人均播放时长+0.109%,

- 人均播放数+0.114%,

- 人均UVV+0.103%,

- 人均session个数+0.124%,

- 人均完播数+0.123%

04

总结与展望

04_总结与展望

1. 维稳是重中之重

- a. 内存泄漏问题
 - i. 提升问题发现准确率和消费效率
 - ii. 大盘OOM归因准确率提升
- b. 内存问题进一步左移至开发阶段
- c. 低内存模式，运行时内存降级持续落地

2. 性能侧持续发力

- a. 性能侧指标和工具建设及完善
 - i. 增强内存性能问题的可观测性
- b. 通用优化持续探索
 - i. 从内存分配器的主要目标出发为抖音和不同机型做定制性优化

3. 业务侧合作和活动保障

- a. 固定路径制定内存测业务准入准出标准
- b. 加强拍摄、直播等热点场景内存优化合作
- c. 内存优化结合业务场景精细化落地



THANKS.